Pinelabs Fin

The single end-to-end reference for the app: what it does, how it is wired, how to configure it, how to debug it, and what to watch out for.



Table of contents

1. What this app is
2. Why it exists
3. The two payment flows
4. Architecture overview
5. Key doctypes
6. File map
7. Installation & first-time configuration
8. Day-to-day use
9. Customer contact resolution
10. Terminal routing — single vs mapping
11. Scheduler & reconciliation
12. Webhook handler
13. Developer API
14. Logging & debugging
15. Limitations
16. Troubleshooting
17. Dependencies & environment
18. License

1. What this app is

`pinelabs_fin` is a Frappe / ERPNext app that integrates Pine Labs payments into ERPNext. It adds two buttons to "payable" documents (Sales Invoice, POS Invoice, Payment Request, etc.):

- **Pay on Machine** — routes a payment to a physical Pine Labs Cloud terminal (card / UPI on the device).
- **Send Payment Link** — generates a Plural hosted payment link covering every method enabled in the Plural dashboard (card, UPI, net banking, wallets). The customer pays via SMS / Email link.

Once payment completes, the app creates the ERPNext **Payment Entry** against the source document automatically — no manual reconciliation.

The app is designed to be **marketplace-clean**: zero custom fields on ERPNext core doctypes. All configuration lives in `Pinelabs Settings`; all per-transaction state lives in the new `Pinelabs Transaction` doctype, linked via Dynamic Link.

---

2. Why it exists

Merchants using ERPNext + Pine Labs hardware (or merchants using ERPNext who need Plural Pay-by-Link to collect online) previously had to:

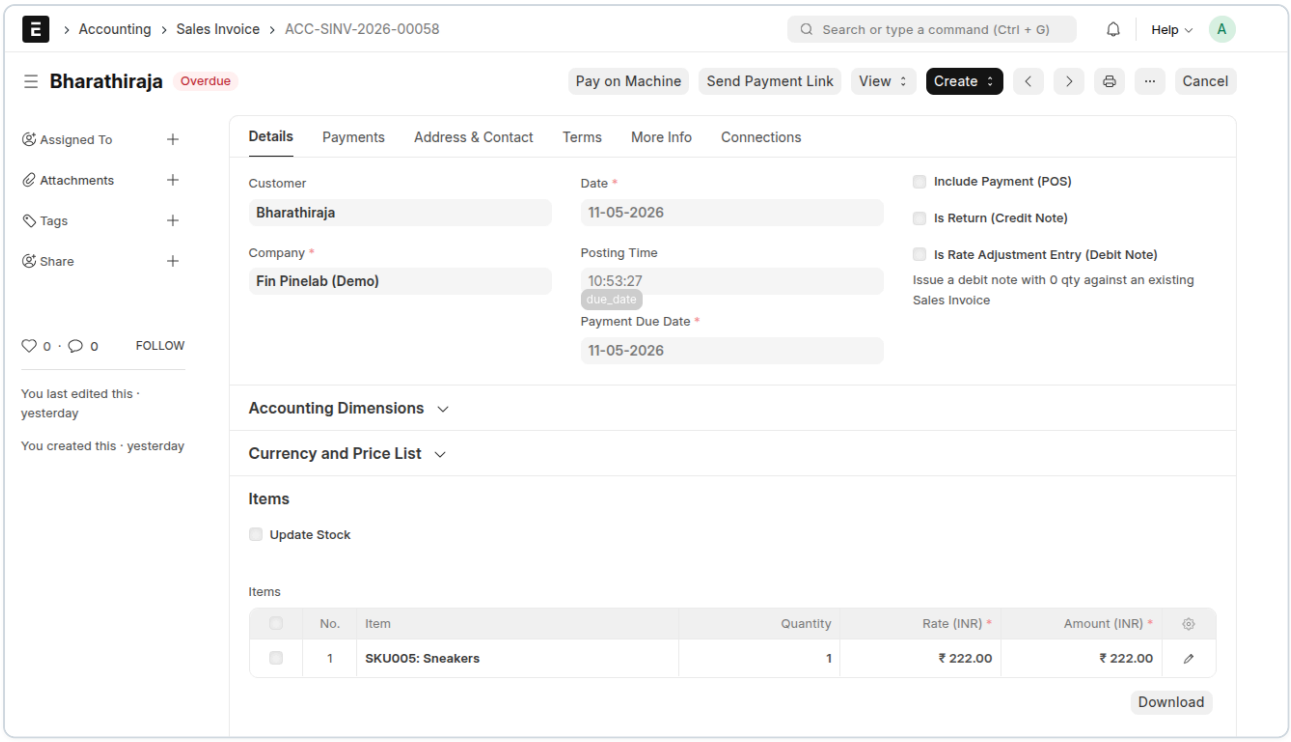
- manually copy invoice references between systems,
- manually mark invoices paid in ERPNext after a customer paid online,
- reconcile Pine Labs settlement reports back to invoices by hand.

This app removes all three. The merchant clicks one button on the invoice; the app handles the API call, the customer flow, the webhook, and the Payment Entry creation. End-to-end automation, with idempotent webhook handling and a per-minute reconcile cron as a backstop.

---

3. The two payment flows

Flow	Button	Source of completion	Customer action
Terminal	Pay on Machine	Foreground 3-second poll + realtime + 1-minute reconcile cron	Tap card / scan UPI on the physical device
Pay-by-Link	Send Payment Link	Webhook from Plural (push) + 1-minute reconcile cron (pull, fallback)	Open SMS/Email link → pick any method on Plural's hosted page



Both flows funnel into the same `Pinelabs Transaction` record and produce the same end result (a submitted Payment Entry).

---

4. Architecture overview

```
ERPNext doc (Sales Invoice, …)
        │  click Pay on Machine / Send Payment Link
        ▼
public/js/pinelabs_buttons.js   ←— eligibility from Pinelabs Settings.payable_do
        │
        ▼
api/terminal.py            api/plural.py            (both create a)
        │                        │                  Pinelabs Transaction
        ▼                        ▼                  (status state machine,
api/pinelabs_client.py     api/plural_client.py     row-locked transitions)
   Pine Labs Cloud           Plural REST + OAuth
        │                        │
   foreground poll          Plural webhook ─► api/webhook.py ─► mark_success/fai
        +                        +
   1-min cron               1-min cron (api/plural.reconcile_pending_plural_payme
        │                        │
        └────────────┬───────────┘
                     ▼
            api/transaction.py
            └── creates ERPNext Payment Entry (idempotent)
```

Single source of truth — `Pinelabs Transaction`

Every payment attempt creates a `Pinelabs Transaction` record. It is polymorphic: a Dynamic Link (`reference_doctype` + `reference_name`) points to the source document.

Status state machine:

```
INITIATED ──► PENDING ──► SUCCESS
                      ├──► FAILED
                      └──► CANCELLED
```

No silent transitions. All transitions go through `api/transaction.py` . Every transition first calls `_lock_and_reload(name)` which issues `SELECT … FOR UPDATE` to take a row-level lock — this serialises concurrent finalize attempts (webhook + cron + foreground poll all racing).

`Mode of Payment` is the source of truth for enabled methods

The install patch seeds four ERPNext Mode of Payment records, each with 4 custom fields under a "Pine Labs Integration" section:

Mode of Payment	flow_type	terminal_code	plural_method
PineLabs - Card	Both	1	CARD
PineLabs - UPI	Both	11	UPI
PineLabs - NetBanking	Plural	–	NB
PineLabs - Wallet	Plural	–	WALLET

The "Pay on Machine" dialog only lists modes where `pinelabs_enabled = 1` AND `pinelabs_flow_type IN ('Terminal', 'Both')` . Disabling a mode hides it from the dialog instantly — no code change.

Strict Payment Entry creation rules

A PE is created ONLY by three paths:

1. **Terminal success** — `api/terminal.py` after foreground poll sees SUCCESS.
2. **Webhook PAID** — `api/webhook.py.handle_webhook` → `mark_success` .
3. **Cron status check** — `api/plural.refresh_payment_link_status` when status flips to PROCESSED.

All three funnel through `api/transaction.py` and are idempotent:

- Bail if a PE with `reference_no = transaction.payment_id` already exists.
- Bail if `transaction.payment_entry` is already populated.

- Use `erpnext.accounts.doctype.payment_entry.payment_entry.get_payment_entry` to build the PE — the same helper ERPNext itself uses.

Zero custom fields on ERPNext core doctypes

No fields are added to Sales Invoice, POS Invoice, Payment Entry, Customer, or Contact. The only custom fields the app installs are on Mode of Payment, which is a configuration doctype, not a transactional one.

---

5. Key doctypes

5.1 Pinelabs Settings (Single)

All app configuration. Form sections (in order):

Section	Contents
Basic Setup	Master `enabled` toggle
Machines	Machines child table + Terminal Routing Mode (Single Machine / Mapping)
Machine Mappings	Routing rules (visible only when Routing Mode = Mapping)
Technical	API Endpoints overrides + Auto Cancel Duration
Plural API Configuration	Enable toggle, Client ID / Secret, Webhook Secret, Base URL, Link Expiry, Last Sync, Test Connection button
Payable Documents	Per-doctype eligibility + per-button toggles + docstatus gating + customer-contact mapping

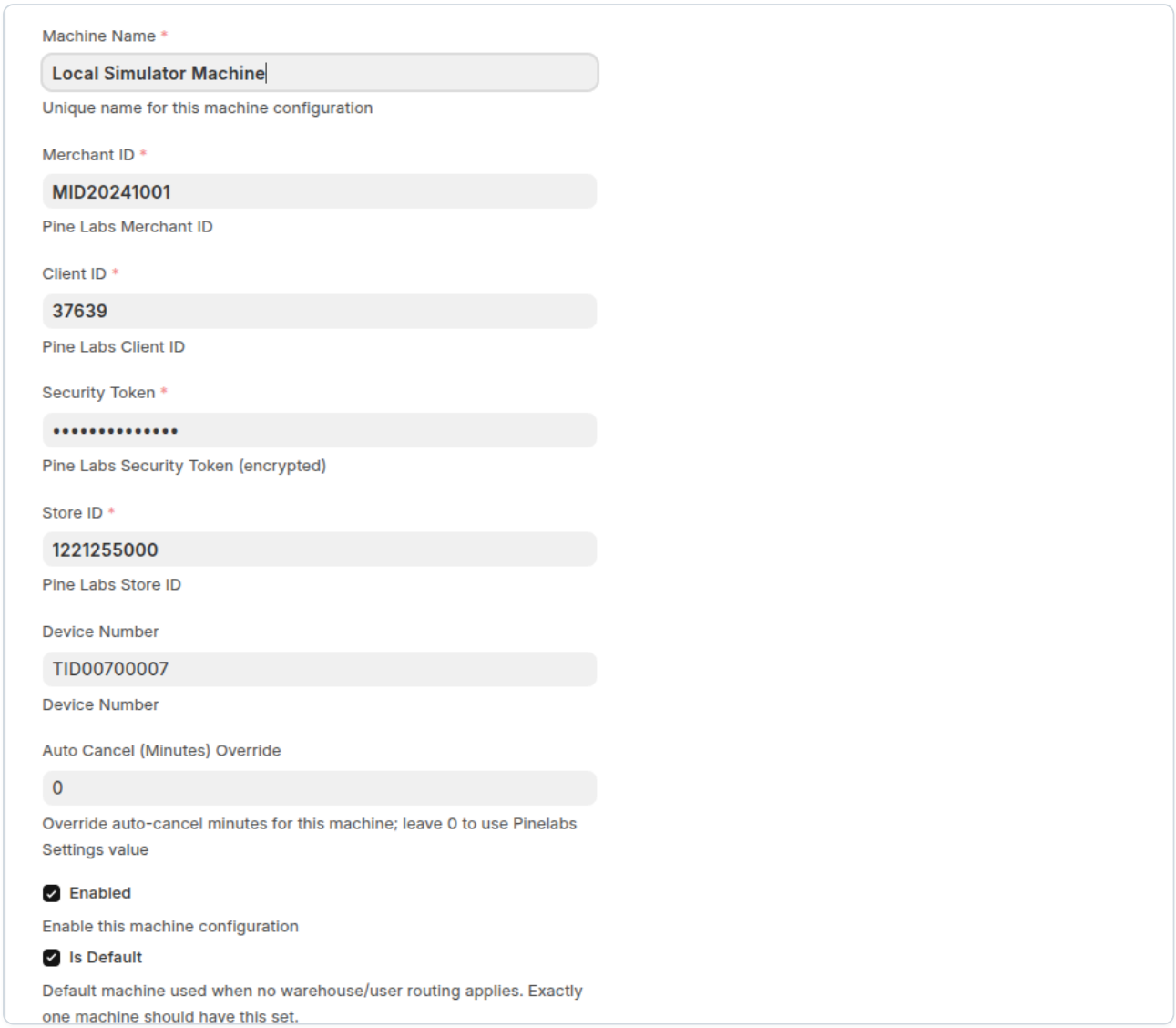
Validation enforced in pinelabs_settings.py:

- At most one machine may be `Is Default`. Empty machines table is OK.
- Plural credentials + Webhook Secret required when payment links enabled. Webhook secret must be ≥ 16 characters.
- Each Payable Doctype row needs `amount_field`. If "Create Payment Entry" is unchecked, both `status_field` and `paid_status_value` are required.
- Customer Contact Field Mapping: when `fetch_from` is set, dependent fields must be populated.

`on_update`: busts the cached Plural OAuth token if any of `plural_client_id` / `plural_client_secret` / `plural_base_url` changed.

5.2 Pinelabs Machine Config (child of `Pinelabs Settings.machines`)

One row per physical Pine Labs Cloud terminal.

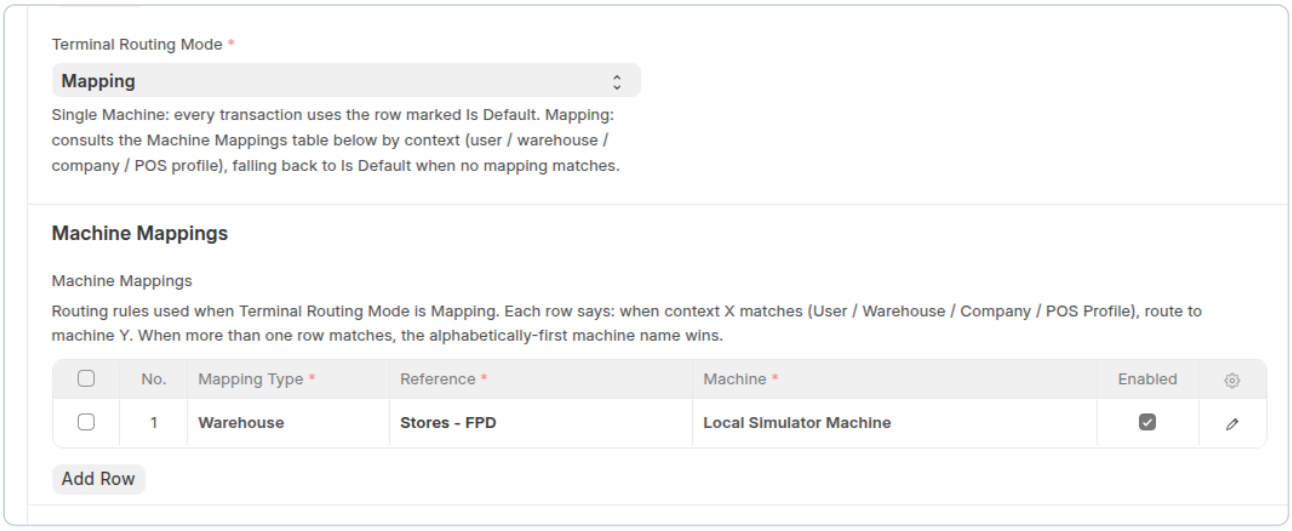


Fields: `machine_name`, `client_id`, `security_token`, `merchant_id`, `store_id`, `terminal_id`, `device_number`, `enabled`, `is_default`, `auto_cancel_duration_in_minutes`.

The `is_default` flag is the fallback when Routing Mode = Single Machine AND when Mapping Mode finds no match.

5.3 Pinelabs Machine Mapping (child of `Pinelabs Settings.machine_mappings`)

Routing rules. Each row maps a context to a machine.



Fields: `machine`, `mapping_type` (User / Warehouse / Company / POS Profile), `reference_doctype`, `reference_name`, `is_enabled`.

Used only when Routing Mode = Mapping. When more than one row matches the current context, the alphabetically-first machine name wins.

5.4 Pinelabs Payable Doctype (child of `Pinelabs Settings.payable_doctypes`)

One row per ERPNext doctype that can be paid via PineLabs.

DocType *

Sales Invoice

Source DocType that should show the Pine Labs payment buttons (e.g. Sales Invoice).

Show on docstatus *

1

Use 0 for Drafts, 1 for Submitted, or 0,1 for both. Cancelled documents are always excluded.

- [x] **Enabled**
- [x] **Pay on Machine**

Show the 'Pay on Machine' button on this doctype. Uncheck to hide the terminal-payment button while leaving the row enabled.

- [x] **Send Payment Link**

Show the 'Send Payment Link' button on this doctype. Uncheck to hide the Pay-by-Link button while leaving the row enabled.

Amount Field *

base_grand_total — Grand Total (Company Currency)

Field on the source doctype that holds the payable amount (e.g. outstanding_amount, grand_total).

Status Field

status — Status

Field on the source doctype to update after a successful payment (used only when Create Payment Entry is unchecked).

Paid Status Value

Paid

Value to write into Status Field after a successful payment (e.g. Paid, Completed, Submitted).

Transaction Reference Field

Optional field on the source doctype where the Pinelabs Transaction id (PLT-...) is stored after success.

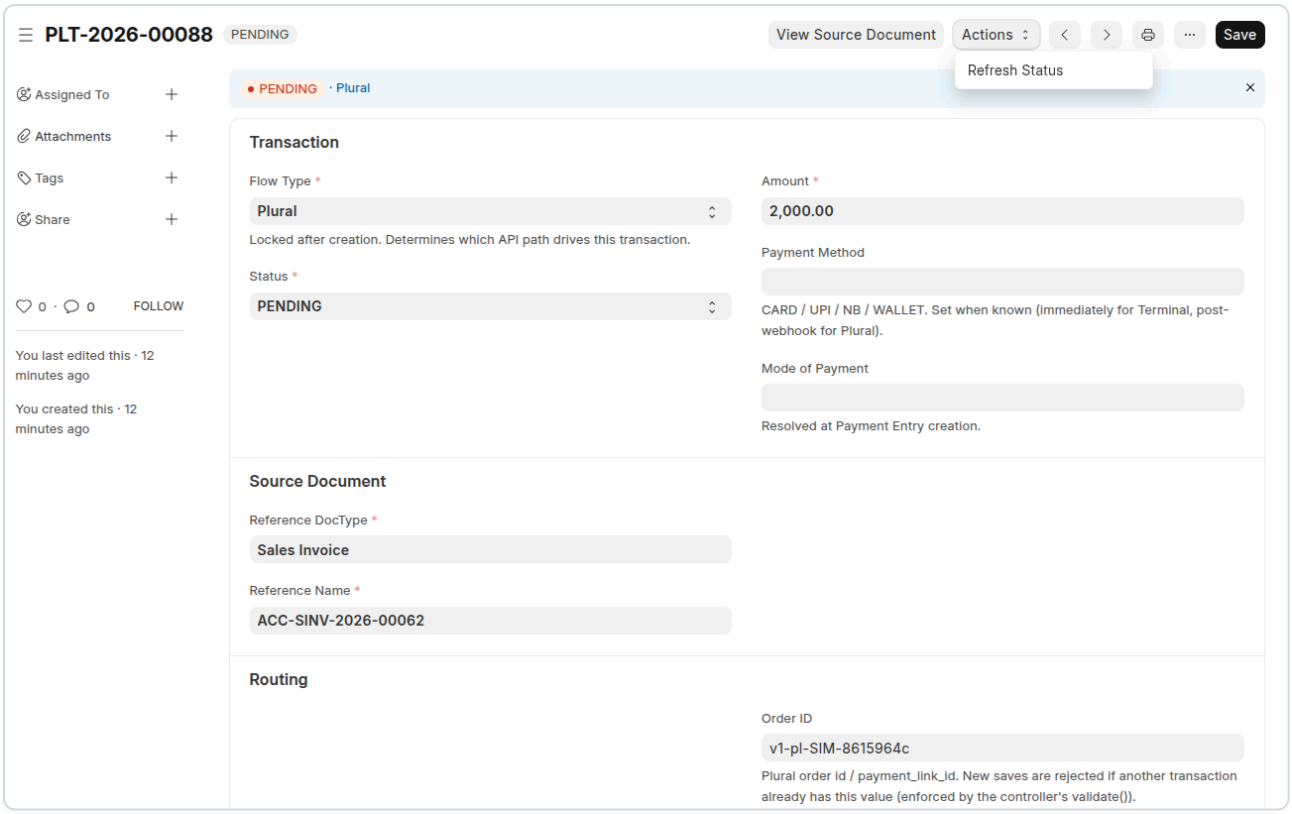
- [x] **Create Payment Entry**

If checked, a standard ERPNext Payment Entry is created on success. If unchecked, only the Status Field on the source doctype is updated.

Field	Purpose
`doctype_name`	Source doctype (Link to DocType)
`is_enabled`	Master toggle for the row
`show_on_docstatus`	"0" / "1" / "0,1" — parsed into a Set of allowed docstatus values. Cancelled (`docstatus=2`) is never accepted
`enable_pay_on_machine`	Show / hide the "Pay on Machine" button on this doctype
`enable_send_payment_link`	Show / hide the "Send Payment Link" button on this doctype
`amount_field`	**Required.** Currency / Float / Int / Percent field on the source doc that holds the payable amount
`status_field` + `paid_status_value`	Used when `finalize_via_payment_entry = 0`
`link_payment_entry_field`	Optional field to write the PE name back into the source doc
`finalize_via_payment_entry`	When unticked, the app writes `status_field = paid_status_value` instead of creating a PE
Customer Contact Field Mapping	See section 9

5.5 Pinelabs Transaction (standalone, one-per-attempt)

All transaction state.



Fields:

```
reference_doctype, reference_name (Dynamic Link)
flow_type (Terminal / Plural)
payment_method (CARD / UPI / NB / WALLET)
amount, status (INITIATED / PENDING / SUCCESS / FAILED / CANCELLED)
machine (link to Machine Config — terminal only)
order_id (Plural's order_id / payment_link_id)
payment_id (Pine Labs payment reference; written to PE.reference_no)
mode_of_payment (resolved at PE time)
payment_entry (link to PE; populated after success)
request_payload, response_payload (JSON, for debugging)
error_message
created_at, completed_at
```

---

6. File map

```
hooks.py                  — Frappe lifecycle hooks (scheduler cron, doc events)
patches.txt               — Install + migration patches

api/                      — HTTP / orchestration
  transaction.py          — Core state machine + PE creation (single source of tr
  terminal.py             — Cloud terminal flow + reconcile_pending_terminal_paym
  plural.py               — Plural orchestration + reconcile_pending_plural_payme
  plural_client.py        — Plural HTTP client (OAuth, payment-link create / get)
  pinelabs_client.py      — Pine Labs Cloud HTTP client (terminal)
  webhook.py              — Plural webhook receiver (HMAC validation, dispatch)
  customer_contact.py     — Resolves customer email / mobile from Payable Doctype
  conflict_guard.py       — Validate-hook on Payment Entry; blocks manual PE when
                             Pinelabs Transaction is INITIATED / PENDING
  config.py               — test_plural_connection (Test Connection button)
  test_phase1.py          — Test suite

pinelabs_fin/doctype/     — Frappe doctypes
  pinelabs_settings/          — Singleton + controller + form JS
  pinelabs_machine_config/    — Cloud terminal credentials child row
  pinelabs_machine_mapping/   — Routing rule child row
  pinelabs_payable_doctype/   — Eligible doctype child row + Contact Field Mappin
  pinelabs_transaction/       — Standalone transaction record + form JS
  pinelabs_bank/              — NB bank list

patches/
  install_pinelabs_modes.py            — Seed 4 PineLabs Modes of Payment +
                                         install custom fields on Mode of Payme
  migrate_machine_warehouse_to_mapping.py — Phase 2 migration of legacy warehous
  redesign_plural_settings_fields.py   — Migrate renamed Plural Settings fields
  seed_pinelabs_banks.py               — Seed Pinelabs Bank rows

public/js/
  pinelabs_buttons.js     — Injects "Pay on Machine" + "Send Payment Link"
                            buttons. Loaded via hooks.app_include_js
```

A subtle Frappe quirk in the JS

`frappe.db.get_list("Pinelabs Payable Doctype", { fields: […] })` silently drops Link and most custom fields when the doctype is a child table — every row comes back with `name` populated and the requested user-defined fields as `undefined`.

The JS works around this by calling `frappe.client.get` on the `Pinelabs Settings` Single and iterating `doc.payable_doctypes`. Child rows arrive fully hydrated. **Do not "fix" this back to**

`get_list` — that's how per-row toggles silently stop working.

Button render gating (four axes; all must pass)

For each Payable Doctype row the JS evaluates:

1. `row.is_enabled == 1`
2. `frm.doc.docstatus ∈ parsed(show_on_docstatus)`
3. `outstanding_amount(frm) > 0` (reads `outstanding_amount` → `grand_total` → `amount` → `paid_amount`; first non-null wins)
4. Per-button: `row.enable_pay_on_machine` / `row.enable_send_payment_link`

---

7. Installation & first-time configuration

7.1 Install

```
cd ~/frappe-bench
bench get-app https://github.com/<owner>/pinelabs_fin
bench --site <your-site> install-app pinelabs_fin
bench --site <your-site> migrate
bench build
bench restart
```

`bench migrate` runs `install_pinelabs_modes.py` (seeds the 4 Modes of Payment + adds the Mode of Payment custom fields), seeds Pinelabs Banks, and installs the doctypes.

7.2 Settings → Basic Setup + Machines

1. Open **Pinelabs Settings**.
2. Tick **Enabled**.
3. Add at least one row in the Machines table — `machine_name` (any human-friendly label), `client_id`, `security_token`, `merchant_id`, `store_id`, `terminal_id`, `device_number` (from your Pine Labs onboarding email), `enabled = 1`, `is_default = 1`.
4. Set **Terminal Routing Mode**: - *Single Machine* — every transaction uses the `is_default` machine. - *Mapping* — consult Machine Mappings by context; fall back to `is_default` if nothing matches.

7.3 Settings → Plural API (only for Send Payment Link)

1. Tick **Enable Payment Links**.
2. **Plural Client ID** + **Plural Client Secret** (from your Plural dashboard).

3. **Webhook Secret (HMAC)** — must match the secret you set in the Plural dashboard. ≥ 16 characters.
4. **Plural API Base URL** — leave blank for sandbox (auto-fills `https://pluraluat.v2.pinepg.in`). For production: `https://api.pluralpay.in`. `site_config.pine_plural_base_url` overrides this when the field is blank (useful for pointing at a local simulator).
5. **Payment Link Expiry (Minutes)** — default 1440 (24 h).
6. Click **Test Plural Connection** to verify OAuth.
7. Save. The save bumps the OAuth token cache automatically.

7.4 Settings → Payable Documents

For each doctype you want PineLabs to be available on:

1. Add a row in **Payable Doctypes**.
2. `doctype_name = "Sales Invoice"` (or POS Invoice / Payment Request / your custom doctype).
3. **Enabled = 1**.
4. **Show on docstatus = "1"** (Submitted, default) / "0" (Draft) / "0,1" (both). Cancelled docs are always excluded.
5. **Pay on Machine = 1** / **Send Payment Link = 1**. Untick one to hide only that button while keeping the row otherwise active.
6. **Amount Field = "outstanding_amount"** (or `grand_total` / `amount`).
7. If you do NOT want a Payment Entry created (custom doctypes, status-field-only flows): - Untick **Create Payment Entry**. - Set **Status Field** + **Paid Status Value** (e.g. `status` + `Paid`).
8. Configure **Customer Contact Field Mapping** (see section 9).

After saving, hard-reload (Ctrl+Shift+R) any open ERPNext form so `pinelabs_buttons.js` picks up the new gating flags. There is no realtime invalidation on Settings save.

7.5 Plural webhook registration

In your **Plural dashboard**, register the webhook:

URL	`https://<your-erpnext-host>/api/method/pinelabs_fin.api.webhook.handle_webhook`
Method	POST
Header	`X-Verify` = HMAC-SHA256 of the raw request body using your Webhook Secret, lower-case hex

The handler refuses every webhook lacking a valid `X-Verify` (fail-closed).

7.6 Confirm scheduler is enabled

```
bench --site <your-site> scheduler enable
bench --site <your-site> scheduler resume
```

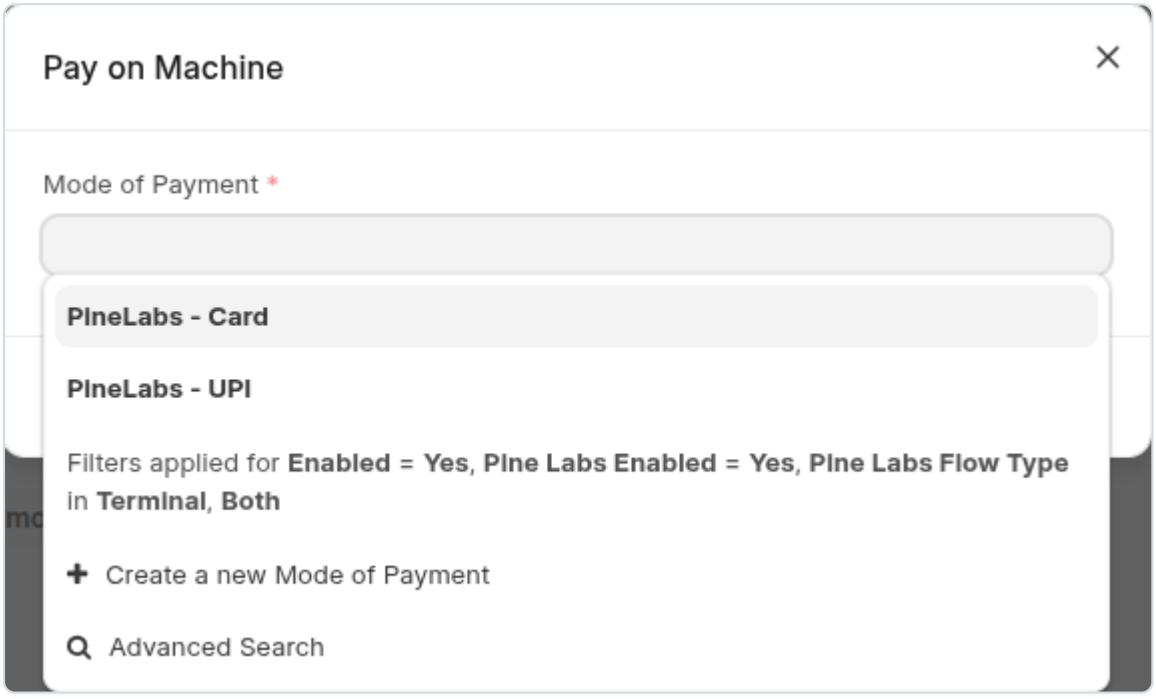
The per-minute reconcile cron is the safety net for missed webhooks.

---

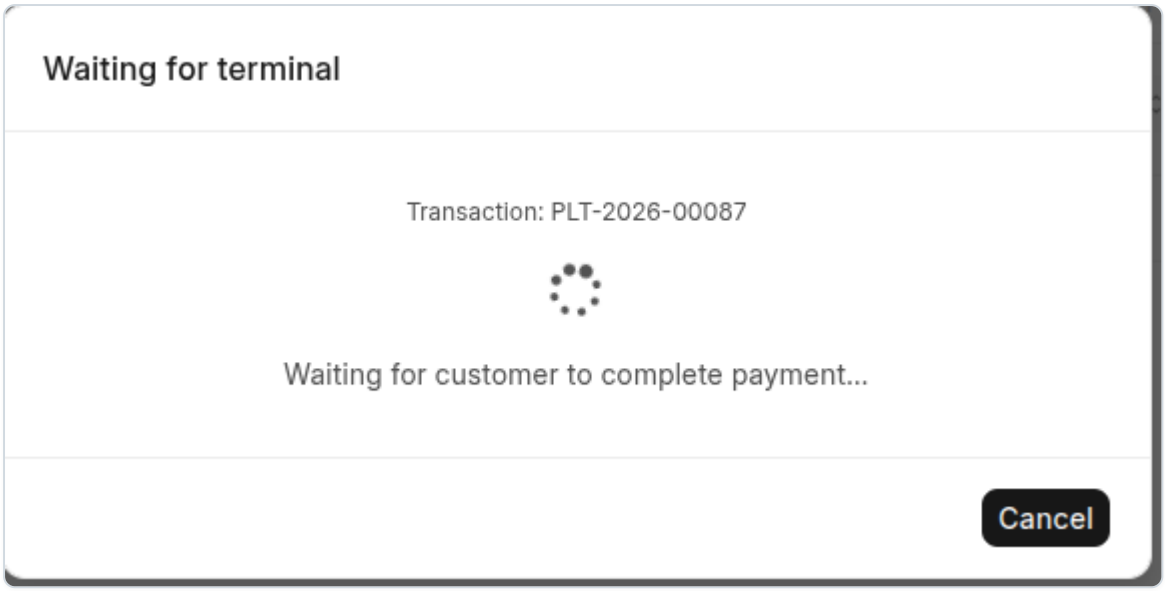
8. Day-to-day use

8.1 Pay on Machine

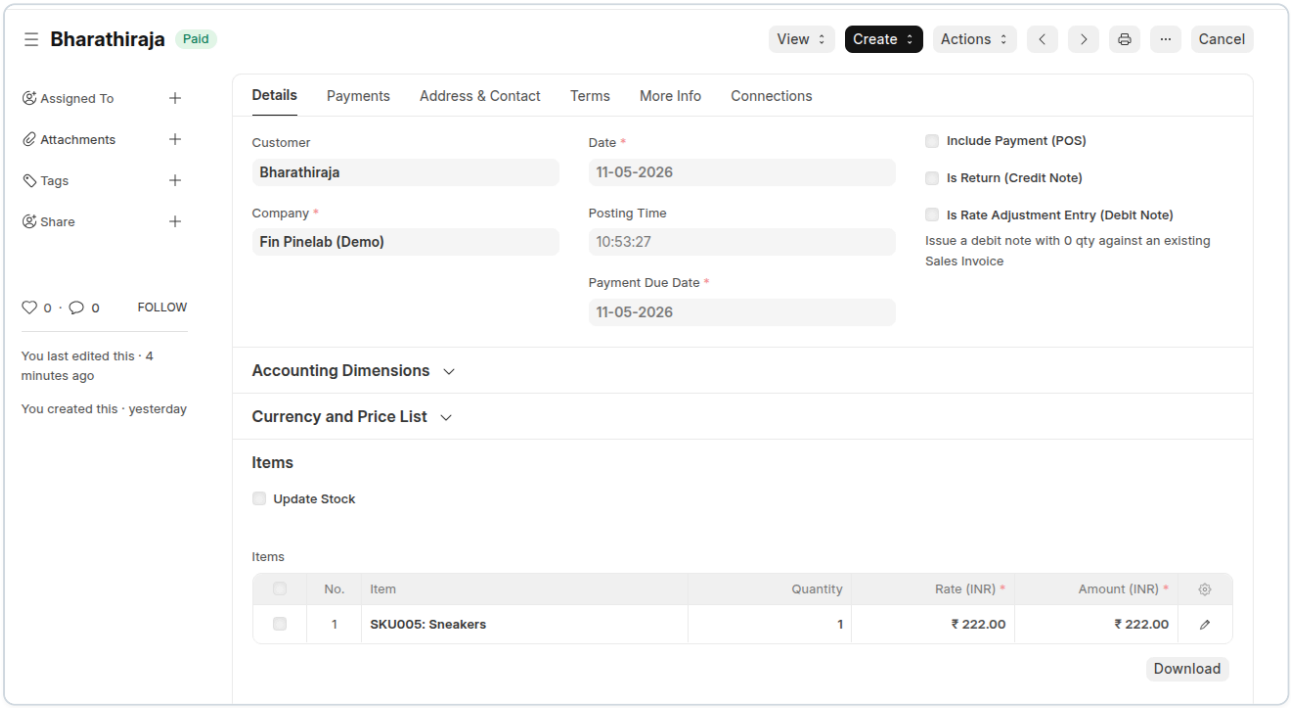
1. Open a submitted Sales Invoice with `outstanding > 0`.
2. Click **Pay on Machine**.
3. Pick a Mode of Payment from the dialog (only Card / UPI for terminal).



4. Dialog opens, shows "Initiated…", polls every 3 seconds.

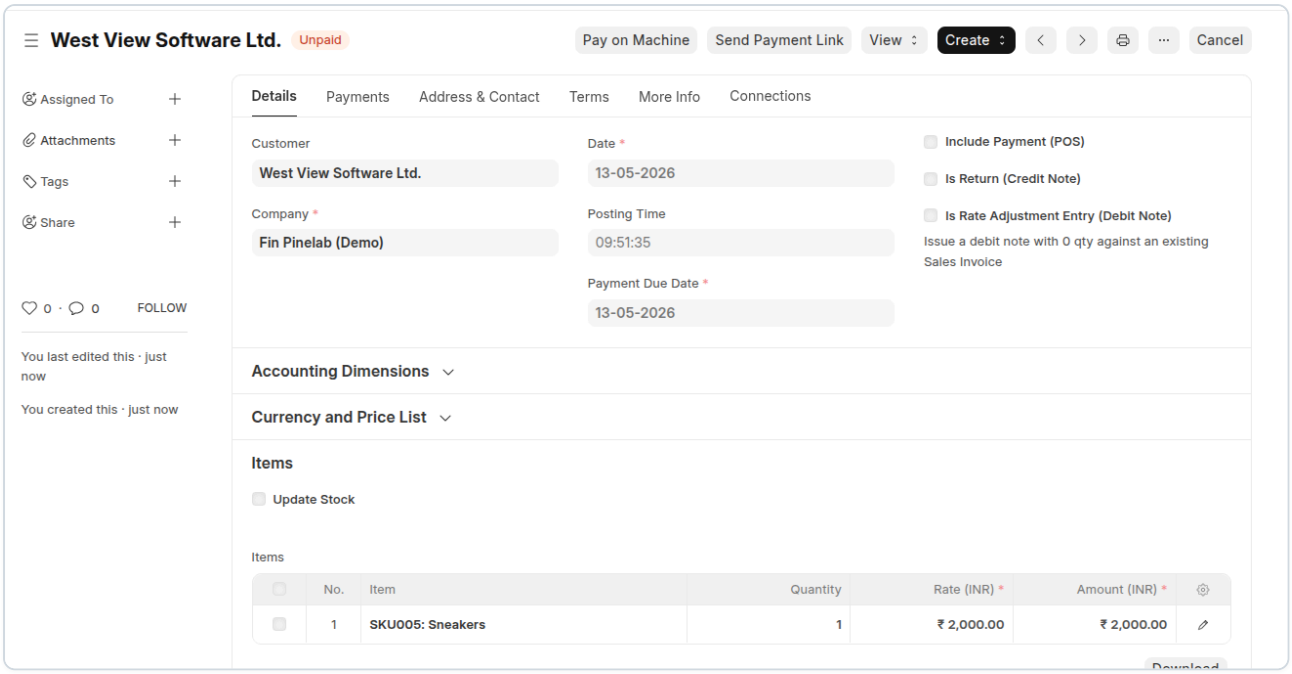


5. Customer taps card / scans UPI on the device.
6. Dialog flips to "Success" within seconds; Payment Entry is created and submitted automatically.

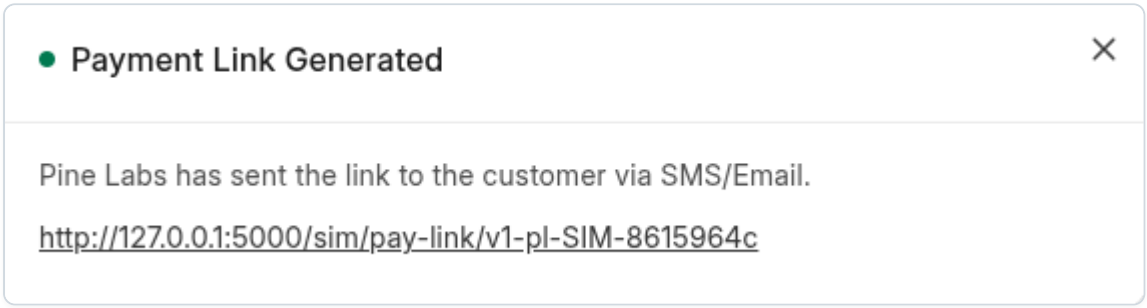


8.2 Send Payment Link

1. Open a submitted Sales Invoice with `outstanding > 0`. The customer must have email OR mobile available via the Contact Field Mapping.
2. Click **Send Payment Link**.



3. No dialog — the link is generated immediately and a toast shows the URL.



4. Plural sends SMS / Email with the link.
5. Customer opens it and pays via any method enabled in the Plural dashboard.
6. Plural fires the webhook to your app. Pinelabs Transaction status flips to SUCCESS; Payment Entry is created.

8.3 Manual status refresh (admin troubleshooting)

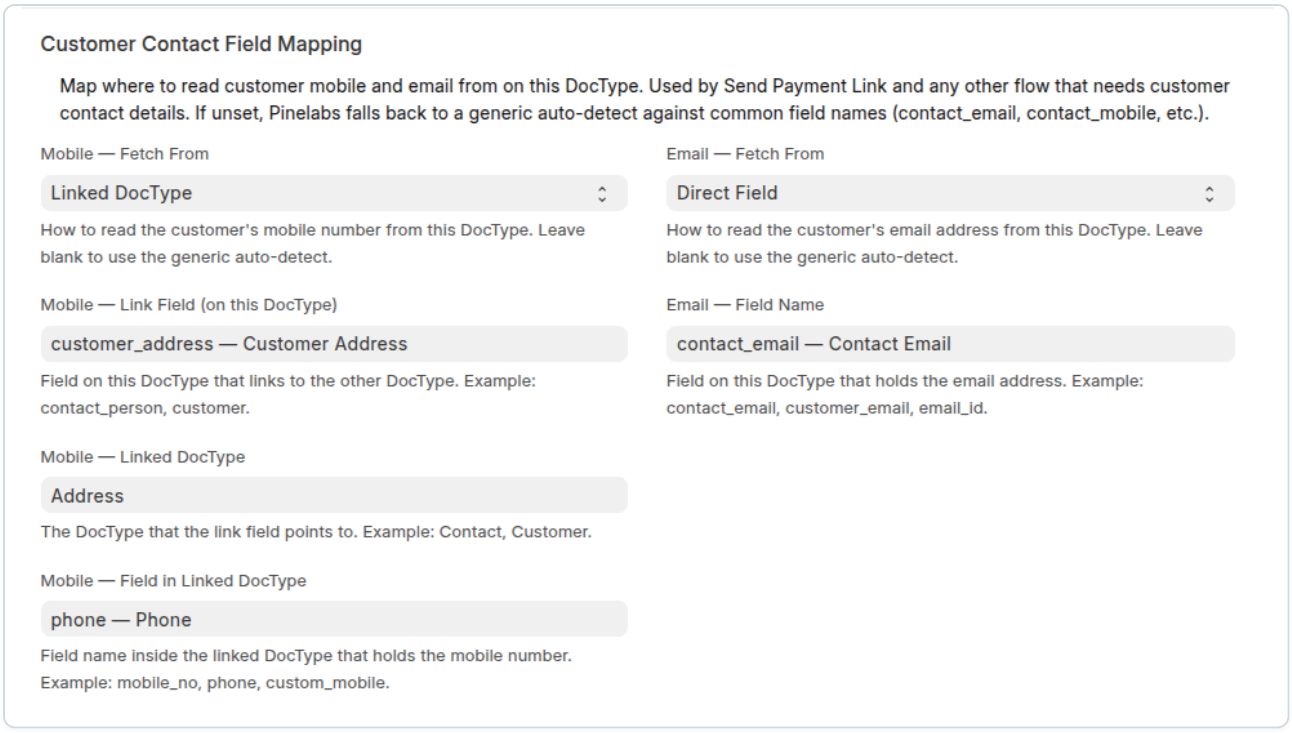
On any Pinelabs Transaction record in PENDING:

Click **Refresh Status** → calls the gateway's status API directly → updates the record + creates the Payment Entry if PROCESSED. Same path the per-minute cron uses, just on demand.

---

9. Customer contact resolution

Plural requires customer `email_id` and `mobile_number` to deliver the SMS / Email link. The app resolves them in this priority order:



1. **Per-doctype config** in Pinelabs Settings → Payable Doctypes → Customer Contact Field Mapping (`mobile_fetch_from` / `email_fetch_from`): - *Direct Field* — read a named field on the source doc. - *Linked DocType* — follow a Link / Dynamic Link field on the source doc to a target doctype, then read a named field there.
2. **Well-known direct fields on the source doc:** - Mobile: `contact_mobile` / `mobile_no` / `phone` - Email: `contact_email` / `email_id` / `email`
3. **Linked Contact doctype** (default contact for the customer).

If after all three priorities both are empty, the button throws a clear error and prints a resolver trace (which mapping was tried, which raw value was read) so you can fix the mapping.

The resolver lives in api/customer_contact.py.

---

10. Terminal routing — single vs mapping

10.1 Single Machine mode (default)

Every terminal payment uses the `is_default` machine row. Simplest case.

10.2 Mapping mode

Each transaction is routed to a machine by context. The router (`resolve_machine_for_context` in `pinelabs_settings.py`) extracts contexts from the source doc + session:

- `("User", frappe.session.user)` — always, except for Guest
- `("Warehouse", doc.set_warehouse or doc.warehouse)`

- `("Company", doc.company)`
- `("POS Profile", doc.pos_profile)`
- `("Warehouse", item.warehouse)` — for each unique items row

Walks `Pinelabs Settings.machine_mappings` (in-memory, not a DB query — unsaved edits are honoured), filtered by `is_enabled` and matching `(mapping_type, reference_name)`. Sorted alphabetically by machine name — first row wins.

Fallback — if no mapping matches, returns the `is_default` machine.

---

11. Scheduler & reconciliation

Two reconcile jobs run every minute via `hooks.scheduler_events["cron"]`:

Job	Catches
`terminal.reconcile_pending_terminal_payments`	Users who closed the terminal-flow dialog before the foreground poll saw a terminal state
`plural.reconcile_pending_plural_payments`	Missed webhooks (no public URL, dropped delivery, secret mismatch, …)

Both route through `mark_success` / `mark_failed`, which are idempotent on terminal states.

---

12. Webhook handler

Endpoint: `POST /api/method/pinelabs_fin.api.webhook.handle_webhook` **Module:** api/webhook.py **Decorator:** `@frappe.whitelist(allow_guest=True)`

HTTP contract

Code	Meaning
200	Processed, or already-processed (idempotent retry)
400	Missing reference / unparseable JSON
401	Missing / invalid `X-Verify`, OR webhook secret unconfigured (**fail-closed**)
404	`order_id` does not match any Pinelabs Transaction (Plural will not retry)
405	GET (or any non-POST)
500	System error (Plural will retry)

Signature

HMAC-SHA256 over the **raw** request body (no re-serialisation), using `Pinelabs Settings → Webhook Secret` as the key, lower-case hex, constant-time compared to the `X-Verify` header.

Routing

Looks up `Pinelabs Transaction` by `order_id`, then `payment_id`, then `name`. First match wins.

Classification

PAID events / statuses → `mark_success`. FAILED events / statuses → `mark_failed`. Unknown event types are recorded in `response_payload` for debugging without changing status.

Event dedup

A `Pinelabs Webhook Event` row is inserted per delivery with a UNIQUE constraint on `event_id`. A redelivery races and one insert wins — the other surfaces as a duplicate and is returned 200 OK without re-running the handler.

---

13. Developer API

Three whitelisted Python entry points — callable from JS or REST — and two JS helpers on `frappe.pinelabs.*` that wrap the API and open the same UI the built-in buttons use.

Python

```
# Pay on Machine
pinelabs_fin.api.terminal.initiate_terminal_payment(
    reference_doctype,    # required — e.g. "Sales Invoice"
    reference_name,       # required — e.g. "ACC-SINV-2026-00054"
    mode_of_payment,      # required — a PineLabs-enabled Mode of Payment
    amount=None,          # optional — defaults to source doc's outstanding
)

# Send Payment Link
pinelabs_fin.api.plural.initiate_payment_link(
    reference_doctype,        # required
    reference_name,           # required
    flow="all_methods",       # optional — "all_methods" / "upi_only" / "card_on
    expiry_minutes=None,      # optional — defaults to 1440 (24 h)
    customer_email=None,      # optional — inline override; skips mapping lookup
    customer_mobile=None,     # optional — inline override; skips mapping lookup
)

# Poll a Pay-on-Machine transaction
pinelabs_fin.api.terminal.check_terminal_status(
    transaction_name,    # required — the PLT-... from initiate_terminal_payment
)
```

JavaScript helpers

```
frappe.pinelabs.start_terminal_payment(
  { reference_doctype: frm.doctype, reference_name: frm.docname, mode_of_payment
  { on_settled: () => frm.reload_doc() },
);

frappe.pinelabs.start_payment_link(
  { reference_doctype: frm.doctype, reference_name: frm.docname, flow: "all_meth
  { on_link: () => frm.reload_doc() },
);
```

Both helpers open the same modal / popup the built-in buttons use, so a custom button gets the standard UX for free.

---

14. Logging & debugging

14.1 The `bench start` terminal is the primary log channel

Every Pay-by-Link API call prints a labelled block. Labels you'll see:

```
[PineLabs PayByLink] Initiate:
[PineLabs PayByLink] Token Request:    /  Token Response:    /  Token ERROR:
[PineLabs PayByLink] Link Request:     /  Link Response:     /  Link ERROR:
[PineLabs PayByLink] Status Request:   /  Status Response:   /  Status ERROR:
[PineLabs PayByLink] Webhook Received:
[PineLabs PayByLink] Webhook Decision:
[PineLabs PayByLink] Initiate Result:
```

Capture for later analysis:

```
bench start 2>&1 | tee /tmp/bench-start.log
grep "PineLabs PayByLink" /tmp/bench-start.log
```

14.2 `frappe.logger("pinelabs_paybylink")` is unreliable

In some bench configurations the file `~/<bench>/logs/pinelabs_paybylink.log` is created but never written to — Frappe's named-logger handler routing depends on the bench config. **Trust the `bench start` terminal first.**

14.3 Standard Frappe logs (`~/<bench>/logs/`)

File	Contents
`web.log`	gunicorn access log (no app-level lines)
`worker.log`	cron output
`frappe.log`	App errors via `frappe.log_error`
`scheduler.log`	Scheduler heartbeat

14.4 In-app debugging

Open any Pinelabs Transaction record. The form shows:

- `request_payload` — JSON sent to Plural / Pine Labs
- `response_payload` — JSON received back (including webhook body for completed Plural transactions)
- `error_message` — populated when status = FAILED

The form auto-refreshes via realtime when status changes.

---

15. Limitations

- **Refunds** — the app does not initiate refunds. Use Pine Labs' own refund flow on their dashboard, then mirror the change in ERPNext manually (cancel the Payment Entry, etc.).
- **Currency** — only INR is supported. Plural's API doesn't accept other currencies in this release.
- **Automatic Payment Entry creation** — works for Sales Invoice, POS Invoice, and Payment Request. Custom doctypes use the status-field mapping path (set `finalize_via_payment_entry = 0` on the Payable Documents row).
- **Webhook handler is fail-closed on missing secret** — if you forget to set Webhook Secret, every webhook returns 401. Intentional. Set the secret BEFORE any payments go live.
- **`bench start` does not auto-reload Python code** — editing `api/*.py` requires `pkill -f "frappe serve --port <port>"` (honcho respawns) or Ctrl+C + `bench start` again.
- **SELECT FOR UPDATE requires InnoDB** — all Pinelabs Transaction status transitions take a row lock. Don't switch the table to MyISAM.
- **Settings save does not invalidate other open tabs** — other tabs see stale gating flags until they hard-reload.

---

16. Troubleshooting

Symptom	Likely cause	Fix
Buttons don't show on the invoice	Doctype not added to Payable Documents, or row not Enabled, or `outstanding_amount = 0`	Pinelabs Settings → Payable Documents: add / enable the row. Verify amount field
`Plural API: 401 Unauthorized`	Wrong Client ID / Secret, or sandbox creds aimed at production URL	Re-check credentials. Click **Test Plural Connection**
Webhook from Plural returns 401 (payment stays PENDING)	Webhook Secret on the Plural dashboard ≠ the one in Pinelabs Settings	Make them identical (byte-for-byte, ≥ 16 characters)
`No Pinelabs machine resolves for this transaction`	No machine has Is Default ticked, or all machines disabled	Mark exactly one row as Is Default and enabled
`Customer email or mobile is required` (Send Payment Link)	The doctype's Customer Contact Field Mapping doesn't resolve, or the customer record has neither	Fill the customer's email / mobile, or fix the mapping. Error includes a resolver trace
Pinelabs Transaction stuck at PENDING after customer paid	Webhook didn't arrive, or arrived and was rejected	Check Webhook Secret matches Plural dashboard. Check `bench start` for `Webhook Received:`. Per-minute cron picks it up within 60 s if scheduler is enabled
Terminal status stuck at PENDING after card tap	Foreground poll missed the terminal state	Reload the form. The 3-layer defense (poll + realtime + 1-min cron) picks it up. Check `response_payload` on the transaction record
Payment Entry not created after success on a custom doctype	ERPNext's PE builder doesn't support arbitrary doctypes	Untick **Create Payment Entry** on the Payable Documents row and set **Status Field** + **Paid Status Value** instead
405 / 401 in bench-start logs at random times	Curl probes / health checks	405 = GET to POST-only endpoint. 401 = POST to webhook without X-Verify. Both prove the endpoint is reachable

17. Dependencies & environment

Runtime

- Frappe v15
- ERPNext v15
- Python 3.10+
- MariaDB 10.6+ with InnoDB

Production URLs

Plural sandbox	`https://pluraluat.v2.pinepg.in`
Plural production	`https://api.pluralpay.in`
Webhook endpoint	`https://<your-erpnext-host>/api/method/pinelabs_fin.api.webhook.handle_webhook`

Useful commands

```
bench start                                                  # dev server
bench --site <site> migrate                                  # apply schema + pa
bench --site <site> console                                  # Python REPL with
bench --site <site> tail-logs                                # tail site logs
bench --site <site> run-tests --app pinelabs_fin             # full test run
bench --site <site> scheduler enable && scheduler resume     # ensure cron runs
pkill -f "frappe serve --port <port>"                        # reload web code
```

18. License

MIT — see license.txt.

Built with Frappe · by Finstein